

Malicious model to RCE by vocab file load in TransfoXLTokenizer (as well as the code reuse to evade the pickle scanning) in [huggingface/transformers](#)

✓ Valid

Reported on Sep 9th 2023

Description

The huggingface/transformers implements `TransfoXLTokenizer` to automatically load `vocab.pkl` file from the remote repo using the risky `pickle.load` without any restrictions, which make the victim users vulnerable to remote code execution attacks by simply running the demo code `AutoTokenizer.from_pretrained("the repo deployed with malicious vocab.pkl file")`. We understand the huggingface implements a `pickle scanning` to flag the `unsafe file` alert if the `os.system` and many other black-listed modules imported in the `vocab.pkl`. But we can bypass this `unsafe` alert in our attacking repo by launching the real attack in another repo. Saying we have two repos A and B. In the repo A, we just deploy the malicious `vocab.pkl` with the execution of `AutoTokenizer.from_pretrained("B")`, and in the repo B, we deploy the malicious `vocab.pkl` with the OS commands we try to execute. By this way, the repo A looks benign since its `vocab.pkl` do not contain any black-listed imports, but the victim users who fetch the pretrained `TransfoXLTokenizer` models from repo A will be forced to execute the malicious command deployed in repo B. It is worth noting that, the victim users cannot find any tricks for the existence of the repo B in the repo A in huggingface websites.

Proof of Concept

We deploy a PoC repo in <https://huggingface.co/zpbrent/test>, by which you can run the following code to fetch the pretrained `TransfoXLTokenizer` in your local machine:

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("zpbrent/test")
```

and then you can find a `HACKED` file has been created in your machine, which means our malicious OS command `touch HACKED` has been successfully executed.

Note that, in our current PoC demo, you may see the `ValueError: Unknown format` at the end of the code execution (despite this cannot prevent the execution of our malicious commands), this is because we have not embedded our malicious payloads in some normal `vocab.pkl`. To hide

such errors, a possible way is to embed our attacking payload into some typical vocab.pkl for TransfoXL such as from <https://huggingface.co/transfo-xl-wt103> and so on.

Impact

This vulnerability can be exploited to execute arbitrary codes and OS commands in the victim machine who fetch the pretrained repo remotely, and obviously leading to disastrous consequences depending on which commands the malicious model includes.

References

- <https://docs.python.org/3/library/pickle.html>

⚙️ We are processing your report and will contact the **huggingface/transformers** team within 24 hours. 2 years ago

✉️ We have contacted a member of the **huggingface/transformers** team and are waiting to hear back 2 years ago



ℹ️ A **huggingface/transformers** maintainer has marked this vulnerability as informative 2 years ago

Hi! The reporter uses one of our methods that loads a pickle file, passes it as a malicious pickle file, and then reports that it loads the file. We recommend using the safetensors format instead. Thanks!

💰 The disclosure bounty has been dropped ❌

💰 The fix bounty has been dropped ❌

★ The researcher's credibility has not been affected



Peng Zhou commented 2 years ago

Researcher

Hi @huggingface, I think this is indeed not the same as just using your method to load a malicious pickle file by myself. Instead, we can use this vulnerability to create a malicious pretrained models and deploy them in your huggingface websites. By this way, the another users who fetch the models from your supplied `from_pretrained` method may be attacked by this vulnerability automatically. In another words, by this vulnerability we can make your huggingface model websites as the attacking weapon, and execute arbitrary commands in any other huggingface customers who fetch the malicious models. And more importantly, these customers may believe they just download a pretrained models from your side and not be aware of the existent of the attacks. I just go through some of your huggingface model websites, and find some of these models have more than ten thousands downloads each months, meaning this new attacking vector may have a very large number of victims if you do not fix it. Moreover, I know your threat model do not guarantee to protect against the malicious pickle models if the users actively load them (that is why I find more than 10 times of pickle.load in your code base, but just report this one), but this vulnerability is totally different as the victims do not actively download any pickle files. They just fetch the

pretrained models from huggingface model websites and your code in `TransfoXLTTokenizer.from_pretrained` automatically run the `pickle.load` to load the `vocab.pkl`. Further, I do not think we can use `safetensors` format to solve this vulnerability, since we exploit this vulnerability to purposely deploy malicious pickle files in huggingface model websites (and bypass your pickle scanning as well), and the victim users who fetch the crafted pretrained model have no way to not load the pickle files as the code is written in your transformer code base. If you suggest to use the `safetensors` format to avoid this vulnerability, your side should use some `safetensors.load()` function to replace the current `pickle.load()` at the code in line 228 in `tokenization_transfo_xl.py`. I thereby hope you can reconsider your decision for this report, thanks!



A **huggingface/transformers** maintainer commented 2 years ago

Maintainer

Thanks for this report - our open source team implemented a solution that will require users to set an env. variable (`TRUST_REMOTE_CODE`) in order to access `pickle.load` in transformers code. Full details on the PR can be found at <https://github.com/huggingface/transformers/pull/27776>. Please note this PR also addresses <https://huntr.com/bounties/423611ee-7a2a-442a-babb-3ed2f8385c16/>.



Peng Zhou commented 2 years ago

Researcher

Hi @Maintainer, Seems only the Admin of the huntr can reinstate this report. I have contacted them and seem to need your clearance to confirm this report is indeed a security vulnerability, and also give them the CVSS score for validation. Is it possible for your team to clarify these matters here? Many thanks!



A **huggingface/transformers** maintainer commented 2 years ago

Maintainer

Hi @zpbrent, Thanks for reaching out. We've added the comment above, about the solution that will require users to set an env. variable (`TRUST_REMOTE_CODE`) in order to access `pickle.load` in transformers code, per your request. Please note that our recommendation to use the `safetensors` format instead still applies.



Peng Zhou commented 2 years ago

Researcher

Hi HuggingFace @maintainer, Thank you for your feedback. First of all, I fully understand that the HF excludes the `pickle.load` as security vulnerability if it can only be used to load files locally. Second, I also agree with you that the transformer may support some other method to use `safetensors` format for vocab serialization in `transfo-xl` models. And if the `model creators` choose to use the `safetensors` to deploy vocabs in their `transfo-xl` models, the `model users` who download the model from the hub by the official `from_pretrained()` function do not suffer any security threats.

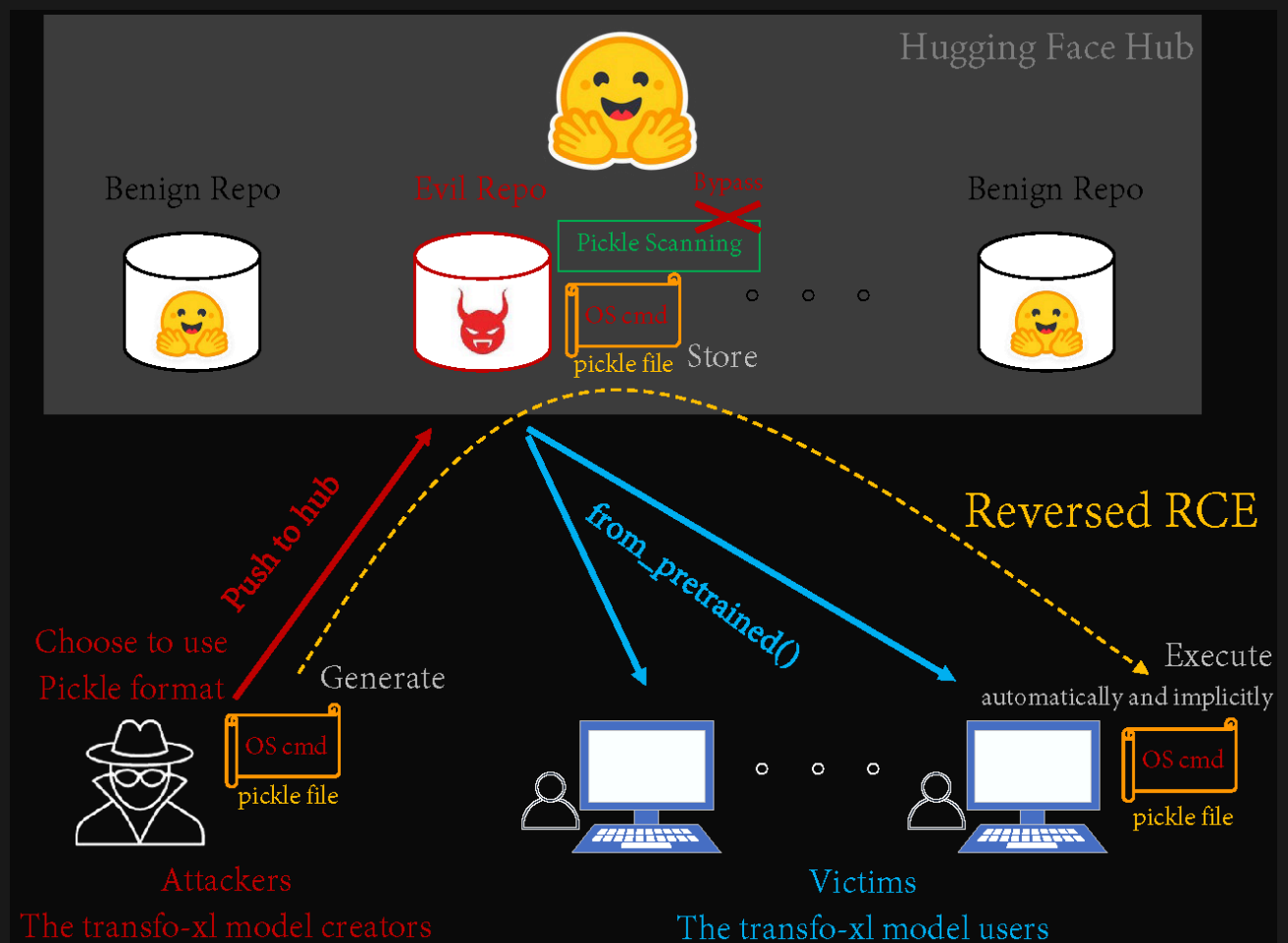
BUT

This `pickle.load` used in the `transfo-xl` model is totally different rather than the local loads. It is exposed to the transformer's official API `from_pretrained()` that is called by the HF users locally but can drive to fetch the remote HF model files and load them **implicitly and automatically**. By combining kinds of tricks to bypass the pickle scanning deployed at

HF, the malicious HF repository can exploit this vulnerability to remotely execute arbitrary commands in the victims' local machines and even to infect pickle malware across the HF accounts.

To my opinion, it seems we still have some **misunderstanding** about the threat model for my report. In fact, we consider the `model creators` as the `attackers` and the `model users` attracted by the attacker's `transfo-xl` models are the `victims`. In this case, the `attackers` can arbitrarily choose to use the pickle format to deliver the vocab files as they take charge of creating the models in the HuggingFace. The `victims` have no way to change the attackers' choice and thus can just download the models in pickle formats if they have been attracted to fetch the model from the hub. I think this is a very practical threat model applicable for the HuggingFace case, because the HuggingFace is designed as an open platform for the purpose to sharing of pre-trained models across the ML communities, all the HF users can freely play both the roles of `model creators` and `model users`.

For a clear description, I drew a picture to show this threat model below:



PoC

We display a **real-world attacking scenario** as the PoC to show how we can exploit this `unsafe pickle.load` from `transfo-xl` models for **reversed RCE** and even **worm infection** over the HuggingFace platform.

We also deployed a **video demo** at <http://zpbrent.github.io/pocs/PoC-Transfo-XLrar> with extracting password `huggingfacetransfoxl` to showcase these steps:

1), we have an `attacker` HuggingFace account `zpbrent` and a `victim` one `pzhoutest` 2), the attacker `zpbrent` deploys a `transfo-xl` model `zpbrent/transfo-xl` for phishing. We understand that HuggingFace launches a **pickle scanning** to alert `unsafe file` for malicious pickle files. But our phishing repo `zpbrent/transfo-xl` can bypass this scanning

via **code reuse** 3), the victim `pzhoutest` did not have the `transfo-xl` model `pzhoutest/transfo-xl` before being attacked, and also no `HACKED` file in the disk. 4), for reversed RCE, the pre-requisite is the victim `pzhoutest` has been attracted by the pre-trained `transfo-xl` model deployed at repo `zpbrent/transfo-xl` . 4.1), the victim finds this repo nothing `unsafe` and thus downloads this pre-trained model using HuggingFace's **official** `from_pretrained` function** `from_pretrained("zpbrent/transfo-xl")` . 4.1), the victim observes the model downloading complete. 4.3), but meanwhile the `HACKED` file has been illegally created in the disk (meaning the success of **reversed RCE**). 5), for worm infection, an additional pre-requisite is the victim `pzhoutest` has already logged in HuggingFace with a `write` permission (`huggingface-cli login --token xxx`) when running the **official** `from_pretrained()` function for downloading. 5.1), a new repo `pzhoutest/transfo-xl` has been implicitly created and uploaded in the victim `pzhoutest` 's account, with the same contents as `zpbrent/transfo-xl` (meaning the success of **worm infection**).

IMPACT

This vulnerability can be exploited to launch the **reversed RCE** as well as the **worm infection** over the HuggingFace hub. This attack can cause disastrous impacts on victims' online accounts and local machines, completely compromising their integrity, confidentiality, and availability. I think HuggingFace respects every single user as a valuable treasure and takes the responsibility to protect any of them. Even worse, we can further exploit this kind of vulnerability to **infect worms** and thus `abuse HuggingFace as an evil weapon` for delivering and propagating pickle malware over ML communities.

To evaluate the severity of this vulnerability's impacts, I think it is possible to compare it with the `Github Submodule Vulnerability (CVE-2018-17456)` <https://github.blog/2018-10-05-git-submodule-vulnerability>. In that case, the attackers can just deploy a malicious Github repo and the victims can be attacked when they clone the Github repos by running the **official command** `git clone --recurse-submodules` or `git submodule update` . To the victims' understanding, they just clone some codes remotely, but in fact, some malicious codes are executed implicitly as well.

The Github case is very similar to our case: the attacker deploys a malicious `transfo-xl` model in the HuggingFace hub and the victims just run the official function `from_pretrained()` to fetch the model. The victims just consider downloading the model but actually, some malicious codes are executed implicitly in the meantime. We can see at <https://nvd.nist.gov/vuln/detail/CVE-2018-17456>, that the NVD evaluates the severity of this Github issue with Critical (9.8). I therefore believe our `transfo-xl` case can receive a similar CVSS as the Github one, but also with respect to the maintainer's choice.

FIX

I think the current fix your open-source team proposed at <https://github.com/huggingface/transformers/pull/27776> addressed this issue by alerting users before the automatic calling of `pickle.load`. By this patch, the users are required to explicitly set an env. variable (`TRUST_REMOTE_CODE`) in order to express their consent to access the `pickle.load` in `transfo-xl` model, hence preventing to load the vocab pickle files by the **official** `from_pretrained()` function **implicitly and automatically**. As the demo code displayed on the HuggingFace webpage has not set `TRUST_REMOTE_CODE=true` in the `from_pretrained()` function by default (see some of the demo codes below), the execution can be paused before the calling of `pickle.load` , displaying alert information to wake up the users' vigilance.

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("zpbrent/transfo-xl")
model = AutoModelForCausalLM.from_pretrained("zpbrent/transfo-xl")
```

However, I do not think the use of `safesensors` can address this issue any way (i.e., the victims have no way to change the attackers' choice for the use of `pickle`). Despite we can recommend the users to fetch models formatting in `safesensors` from the hub, but it is impossible to restrict the HF users solely to that formate. In deed, there is a large number of ML models and libraries still applying `pickle` formats. Especially the **Pytorch** is still using it, I do not think it is possible to get rid of using pickle ML models in a short period.

#THE END I fully understand the ML communities have a **stubborn impression** that ML libraries commonly exclude `pickle.loads` as security vulnerabilities if they are used to load files locally. But this one is **totally different** as it exposes this risky load to the official `from_pretrained()` API which involves the communication with the remote HuggingFace repositories. I think your team have also captured this point and thus proposed a PR to fix it at <https://github.com/huggingface/transformers/pull/27776>.

By these reasons, I ask your kindness to validate this security vulnerability, hence allowing me to collect the bounty for disclosing and reporting these findings to enhance the HuggingFace's security.

Please let me know if I miss any points, or if you have any more questions regarding the Impact analysis.

Thanks!

 **Adam Nygate** modified the CWE from Use of GET Request Method With Sensitive Query Strings to Deserialization of Untrusted Data 2 years ago

★ The researcher has received a minor penalty to their credibility for misclassifying the vulnerability type: -1

✓ **Michelle Habonneau** validated this vulnerability 2 years ago

\$ **zpbrent** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

✓ **Michelle Habonneau** marked this as fixed in 4.36 with commit **1d63b0** 2 years ago

\$ The fix bounty has been dropped ✗

 This vulnerability has now been published 2 years ago

 **Dan McInerney** gave praise 2 years ago

These are excellent explanations and I love the graphics.

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

Ian Swanson commented 2 years ago

[Admin](#)

Nice work!

[Sign in](#) to join this conversation

CVE
CVE-2023-7018
Published

Vulnerability Type
CWE-502: Deserialization of Untrusted Data

Severity
Critical (9.6)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	Required
Scope	Changed
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#) 

Registry
Pypi

Affected Version
4.33.1

Visibility
Public

Status
Fixed

Disclosure Bounty
\$1500

Fix Bounty
\$375

Found by



Peng Zhou

@zpbrent

LIGHTWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)